

Bài 1: Matrix Exponentiation – String Mood Updates

Ý tưởng

Ta mô hình hóa tâm trạng của Limak bằng 2 trạng thái:

- 0: hạnh phúc
- 1: buồn bã

Ban đầu Limak đang hạnh phúc, tức là trạng thái ban đầu là 0.

Mỗi kí tự khi được đọc sẽ biến đổi trạng thái hiện tại sang trạng thái mới. Vì chỉ có 2 trạng thái, ta có thể dùng ma trận 2×2 để biểu diễn sự biến đổi này.

Gọi ma trận của một kí tự là M , trong đó:

```
M[to][from]
```

là số cách để đi từ trạng thái $from$ sang trạng thái to .

Khi có vector trạng thái:

```
dp[0] = số cách để hiện tại Limak hạnh phúc  
dp[1] = số cách để hiện tại Limak buồn bã
```

Sau khi đọc một kí tự, vector mới sẽ là:

```
new_dp = M * dp
```

Ma trận của từng loại kí tự

Kí tự H

Dù trước đó đang hạnh phúc hay buồn bã, sau khi đọc H đều thành hạnh phúc.

Ma trận:

```
1 1  
0 0
```

Kí tự S hoặc D

Dù trước đó đang hạnh phúc hay buồn bã, sau khi đọc **S** hoặc **D** đều thành buồn bã.

Ma trận:

0	0
1	1

Nguyên âm **A, E, I, O, U**

Các kí tự này đảo trạng thái:

- hạnh phúc thành buồn bã
- buồn bã thành hạnh phúc

Ma trận:

0	1
1	0

Các chữ cái khác

Các chữ cái khác không ảnh hưởng gì đến tâm trạng, tức là giữ nguyên trạng thái.

Ma trận:

1	0
0	1

Kí tự **?**

Dấu **?** có thể thay bằng 26 chữ cái in hoa.

Trong 26 chữ cái:

- Có 1 chữ **H** làm Limak hạnh phúc.
- Có 2 chữ **S, D** làm Limak buồn bã.
- Có 5 nguyên âm **A, E, I, O, U** làm đảo trạng thái.
- Còn lại $26 - 1 - 2 - 5 = 18$ chữ giữ nguyên trạng thái.

Xét từ trạng thái hạnh phúc:

- Thành hạnh phúc:
 - chọn **H**: 1 cách
 - chọn chữ giữ nguyên: 18 cách

- tổng: 19 cách
- Thành buồn bã:
 - chọn **S**, **D**: 2 cách
 - chọn nguyên âm: 5 cách
 - tổng: 7 cách

Xét từ trạng thái buồn bã:

- Thành hạnh phúc:
 - chọn **H**: 1 cách
 - chọn nguyên âm: 5 cách
 - tổng: 6 cách
- Thành buồn bã:
 - chọn **S**, **D**: 2 cách
 - chọn chữ giữ nguyên: 18 cách
 - tổng: 20 cách

Vậy ma trận của **?** là:

19 6
7 20

Ghép nhiều kí tự

Giả sử xâu có 3 kí tự:

s1 s2 s3

Gọi ma trận tương ứng là:

M1, M2, M3

Ta đọc **s1** trước, rồi **s2**, rồi **s3**.

Vector cuối cùng là:

M3 * M2 * M1 * dp_ban_đầu

Vì vậy khi gộp hai đoạn trong segment tree:

- Đoạn trái được đọc trước.
- Đoạn phải được đọc sau.

Nếu **L** là ma trận đoạn trái và **R** là ma trận đoạn phải, thì ma trận tổng phải là:

$R * L$

Do đó trong code có dòng:

```
st[id] = mul(st[2 * id + 1], st[2 * id]);
```

Tức là:

ma trận đoạn phải * ma trận đoạn trái

Xử lý cập nhật

Mỗi truy vấn thay đổi một kí tự trong xâu.

Nếu sau mỗi truy vấn ta tính lại toàn bộ xâu thì mất $O(nq)$, không đủ nhanh vì $n, q \leq 200000$.

Ta dùng segment tree.

Mỗi node của segment tree lưu ma trận tổng của một đoạn.

Khi cập nhật một vị trí:

1. Cập nhật ma trận tại lá tương ứng với kí tự mới.
2. Đi ngược lên các node cha.
3. Gộp lại bằng phép nhân ma trận.

Vì ma trận chỉ có kích thước 2×2 , mỗi lần nhân coi như $O(1)$.

Vì sao đáp án là `st[1].c[0][0]`?

Ban đầu Limak hạnh phúc, tức là trạng thái ban đầu là **0**.

Ta cần sau khi đọc hết xâu, Limak vẫn hạnh phúc, tức là trạng thái cuối cũng là **0**.

Ma trận tổng của cả xâu là `st[1]`.

Trong đó:

```
st[1].c[to][from]
```

Vậy số cách đi từ trạng thái ban đầu **0** đến trạng thái cuối **0** là:

```
st[1].c[0][0]
```

Độ phức tạp

Với n là độ dài xâu và q là số truy vấn:

- Xây cây: $O(n)$
- Mỗi truy vấn: $O(\log n)$
- Bộ nhớ: $O(n)$

Code mẫu

```
#pragma GCC optimize("unroll-loops,02,no-stack-protector")
#include <iostream>
#include <string>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <vector>
#include <set>
#include <map>
#include <deque>
#include <stack>
#include <queue>
#include <algorithm>
#include <cassert>
#include <random>
#include <chrono>
#include <iomanip>
#define int long long
#define ii pair<int,int>
#define fi first
#define se second
#define getbit(x,y) (((x)>>(y))&1)
#define turnon(x,y) ((x)|(1<<y))
#define turnof(x,y) ((x)^(1<<y))
#define oo 2000000000000000000
#define pb push_back
#define all(x) x.begin(),x.end()
#define con(mask) mask=(mask-1)&mask
#define Unique(val) val.erase(unique(val.begin(),val.end()),val.end())

#define rand_int mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
#define rand_ll mt19937_64
rnd(chrono::steady_clock::now().time_since_epoch().count());

const int mod = 1e9 + 7;
```

```
using namespace std;

void add(int &x, int y) {
    x += y;
    if(x >= mod) x -= mod;
}

int n, q;
string s;

struct ma {
    int c[2][2];
};

ma mul(ma x, ma y) {
    ma z;

    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            z.c[i][j] = 0;

            for(int k = 0; k < 2; k++) {
                add(z.c[i][j], x.c[i][k] * y.c[k][j] % mod);
            }
        }
    }
    return z;
}

ma cal(char c) {
    ma z;
    if(c == '?') {
        z.c[0][0] = 19;
        z.c[0][1] = 6;
        z.c[1][0] = 7;
        z.c[1][1] = 20;
    }
    else if(c == 'S' || c == 'D') {
        z.c[0][0] = 0;
        z.c[0][1] = 0;
        z.c[1][0] = 1;
        z.c[1][1] = 1;
    }
    else if(c == 'H') {
        z.c[0][0] = 1;
        z.c[0][1] = 1;
        z.c[1][0] = 0;
        z.c[1][1] = 0;
    }
    else if(c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U') {
        z.c[0][0] = 0;
        z.c[0][1] = 1;
        z.c[1][0] = 1;
        z.c[1][1] = 0;
    }
}
```

```
    }
    else {
        z.c[0][0] = 1;
        z.c[0][1] = 0;
        z.c[1][0] = 0;
        z.c[1][1] = 1;
    }
    return z;
}

ma st[4 * 200000 + 5];

void build(int id, int l, int r) {
    if(l == r) {
        st[id] = cal(s[l]);
        return;
    }

    int mid = (l + r) / 2;

    build(2 * id, l, mid);
    build(2 * id + 1, mid + 1, r);

    st[id] = mul(st[2 * id + 1], st[2 * id]);
}

void update(int id, int l, int r, int pos) {
    if(l > pos || r < pos) return;

    if(l == r && l == pos) {
        st[id] = cal(s[pos]);
        return;
    }

    int mid = (l + r) / 2;

    update(2 * id, l, mid, pos);
    update(2 * id + 1, mid + 1, r, pos);

    st[id] = mul(st[2 * id + 1], st[2 * id]);
}

signed main()
{
    #ifndef ONLINE_JUDGE
    freopen("txt.inp", "r", stdin);
    freopen("txt.out", "w", stdout);
    #endif

    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    cin >> n >> q;
```

```

    cin >> s;

    s = " " + s;

    build(1, 1, n);

    cout << st[1].c[0][0] << "\n";

    for(int i = 1; i <= q; i++) {
        int x;
        char c;

        cin >> x >> c;

        s[x] = c;

        update(1, 1, n, x);

        cout << st[1].c[0][0] << "\n";
    }
}

```

Bài 2: Matrix Exponentiation – Count paths queries

Ý tưởng

Ta có một đồ thị có hướng gồm n đỉnh và m cạnh.

Mỗi truy vấn cho 3 số:

s, t, k

Yêu cầu đếm số đường đi từ s đến t có đúng k bước.

Một đường đi được phép đi qua một đỉnh hoặc một cạnh nhiều lần.

Biểu diễn bằng ma trận kề

Ta dùng ma trận kề A .

Nếu có cạnh từ u đến v , ta gán:

$A[v][u] = 1$

Ở đây ta lưu ngược kiểu $A[v][u]$ thay vì $A[u][v]$ để dễ nhân với vector trạng thái.

Gọi:

```
dp[i] = số cách đi từ đỉnh s đến đỉnh i sau một số bước hiện tại
```

Ban đầu:

```
dp[s] = 1
```

vì ta đang đứng ở đỉnh s .

Sau khi đi thêm một bước, với mỗi cạnh $u \rightarrow v$, ta có:

```
new_dp[v] += dp[u]
```

Nếu dùng ma trận $A[v][u]$, công thức trở thành:

```
new_dp[v] = sum(A[v][u] * dp[u])
```

Tức là:

```
new_dp = A * dp
```

Sau đúng k bước:

```
dp = A^k * dp_ban_đầu
```

Vậy đáp án là:

```
dp[t]
```

Vì sao cần lũy thừa ma trận?

Nếu mô phỏng từng bước, mỗi truy vấn có thể mất $O(k * m)$ hoặc $O(k * n^2)$.

Nhưng k có thể lên tới 10^9 , nên không thể duyệt từng bước.

Ta dùng lũy thừa nhị phân.

Ta tiền xử lý các ma trận:

```
a[0] = A
a[1] = A^2
a[2] = A^4
a[3] = A^8
...
```

Công thức:

```
a[i] = a[i - 1] * a[i - 1]
```

Vì $k \leq 10^9$, ta chỉ cần khoảng 32 ma trận.

Trả lời một truy vấn

Với mỗi truy vấn (s, t, k) :

1. Khởi tạo vector dp toàn 0.
2. Gán $dp[s] = 1$.
3. Tách k thành tổng các lũy thừa của 2.
4. Nếu cần dùng 2^{bit} bước, ta nhân vector hiện tại với ma trận $A^{(2^{bit})}$.

Ví dụ:

```
k = 13 = 8 + 4 + 1
```

Khi đó:

```
A^13 = A^8 * A^4 * A^1
```

Ta nhân vector với các ma trận tương ứng.

Trong code:

```
for(int bit = 31; bit >= 0; bit--) {
    if(k >= (1ll << bit)) {
        dp = a[bit] * dp;
        k -= (1ll << bit);
    }
}
```

```
}  
}
```

Sau khi xử lý xong, $dp[i]$ là số đường đi từ s đến i có đúng k bước ban đầu.

Vì vậy đáp án cần in là:

```
dp[t]
```

Vì sao trong code dùng $D.c[v][u] = 1$?

Giả sử có cạnh:

```
u -> v
```

Nếu đang có $dp[u]$ cách để đứng ở u , thì sau một bước, số cách đứng ở v được cộng thêm $dp[u]$.

Tức là:

```
new_dp[v] += dp[u]
```

Công thức nhân ma trận với vector là:

```
new_dp[i] = sum(A[i][j] * dp[j])
```

Muốn $new_dp[v]$ nhận từ $dp[u]$, ta cần:

```
A[v][u] = 1
```

Vì vậy code lưu:

```
D.c[v][u] = 1;
```

Độ phức tạp

Tiền xử lý:

- Nhân hai ma trận $n \times n$ tốn $O(n^3)$.

- Có khoảng 32 ma trận.

Do đó:

$$O(32 * n^3)$$

Mỗi truy vấn:

- Có khoảng 32 bit.
- Mỗi lần nhân ma trận với vector tốn $O(n^2)$.

Do đó:

$$O(32 * n^2)$$

Bộ nhớ:

$$O(32 * n^2)$$

Với $n \leq 200$, cách này chạy được.

Code mẫu

```
#pragma GCC optimize("unroll-loops,02,no-stack-protector")
#include <iostream>
#include <string>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <vector>
#include <set>
#include <map>
#include <deque>
#include <stack>
#include <queue>
#include <algorithm>
#include <cassert>
#include <random>
#include <chrono>
#include <iomanip>
#define int long long
#define ii pair<int,int>
#define fi first
#define se second
#define getbit(x,y) (((x)>>(y))&1)
```

```

#define turnon(x,y) ((x)|(1<<y))
#define turnof(x,y) ((x)^(1<<y))
#define oo 2000000000000000000
#define pb push_back
#define all(x) x.begin(),x.end()
#define con(mask) mask=(mask-1)&mask
#define Unique(val) val.erase(unique(val.begin(),val.end()),val.end())

#define rand_int mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
#define rand_ll mt19937_64
rnd(chrono::steady_clock::now().time_since_epoch().count());

const int mod = 1e9 + 7;
using namespace std;

void add(int &x, int y) {
    x += y;
    if(x >= mod) x -= mod;
}

int n, m, q;

struct ma {
    int c[200][200];
};

vector<ma> a;

ma mul(ma x, ma y) {
    ma z;

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            z.c[i][j] = 0;

            for(int k = 0; k < n; k++) {
                add(z.c[i][j], x.c[i][k] * y.c[k][j] % mod);
            }
        }
    }

    return z;
}

int dp[205], pre[205];

signed main()
{
    #ifndef ONLINE_JUDGE
    freopen("txt.inp","r",stdin);
    freopen("txt.out","w",stdout);
    #endif

```

```
ios_base::sync_with_stdio(0);
cin.tie(0);
cout.tie(0);

cin >> n >> m >> q;

ma D;

for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        D.c[i][j] = 0;
    }
}

for(int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;

    --u;
    --v;

    D.c[v][u] = 1;
}

a.pb(D);

for(int i = 1; i < 32; i++) {
    a.pb(mul(a.back(), a.back()));
}

while(q--) {
    int s, t, k;

    cin >> s >> t >> k;

    --s;
    --t;

    for(int i = 0; i < n; i++) dp[i] = 0;

    dp[s] = 1;

    for(int bit = 31; bit >= 0; bit--) {
        if(k >= (1ll << bit)) {

            for(int i = 0; i < n; i++) {
                pre[i] = dp[i];
                dp[i] = 0;
            }

            for(int i = 0; i < n; i++) {
                for(int j = 0; j < n; j++) {
                    add(dp[i], a[bit].c[i][j] * pre[j] % mod);
                }
            }
        }
    }
}
```

```
        }  
        k -= (1ll << bit);  
    }  
}  
cout << dp[t] << "\n";  
}
```